



HyperNOs: automated and parallel library for neural operators research

Massimiliano Ghiotto¹ 

Received: 12 June 2025 / Accepted: 7 October 2025

© The Author(s) 2025

Abstract

This paper introduces HyperNOs, a PyTorch library designed to streamline and automate the process of exploring neural operators, with a special focus on hyperparameter optimization for comprehensive and exhaustive exploration. In particular, HyperNOs leverages state-of-the-art optimization algorithms and parallel computing implemented in the Ray-tune library to efficiently explore the hyperparameter space of neural operators. We also implement several features for studying neural operators with a user-friendly interface, such as the ability to train the model with a fixed number of parameters or to train the model with multiple datasets and different resolutions. We integrate Fourier neural operators and convolutional neural operators in our library, achieving state-of-the-art results on many representative benchmarks, demonstrating the capabilities of HyperNOs to handle real datasets and modern architectures. The library is designed for ease of use with the provided model and datasets, but also to be extended to use new datasets and custom neural operator architectures.

Keywords Operator learning · Fourier neural operators · Convolutional neural operators · Pytorch · Ray

1 Introduction

In recent years, the field of scientific machine learning has witnessed significant advances, particularly in the development of methods for solving partial differential equations (PDEs) and modelling complex dynamical systems. Traditional numerical methods, such as finite element analysis, isogeometric analysis [12] or spectral methods [4], have long been the cornerstone of computational science and engineering. However, these methods often face challenges in scalability, computational cost, and generalization across different problem settings. In this context, Neural Operators (NOs) have emerged as a powerful paradigm that provides a data-driven approach to learning mappings between infinite-dimensional function spaces, allowing PDEs and other operator-related problems to be solved with unprecedented efficiency and flexibility.

Neural operators, introduced as a generalization of neural networks to infinite-dimensional spaces, are designed to approximate operators that map input functions to output functions.

✉ Massimiliano Ghiotto
massimiliano.ghiotto01@universitadipavia.it

¹ Department of Mathematics, University of Pavia, Via A. Ferrata 5, 27100 Pavia, Italy

Unlike traditional neural networks, which operate on finite-dimensional vectors, neural operators are capable of handling continuous functions, making them particularly suited for tasks such as solving PDEs, modelling physical systems, and performing surrogate modelling in high-dimensional spaces. The foundational work on neural operators, including architectures such as Deep Operator Networks (DeepONets, [25]), Graph-based Neural Operators (GNOs, [21]), and Fourier Neural Operators (FNOs, [22]), has demonstrated their ability to learn solution operators for a wide range of PDEs. Many modifications of the above architectures have been progressively proposed, such as NOMAD [35] as a DeepONet modification, Multiple Graph Neural Operator [20] as a GNO modification, and many FNO modifications such as [23, 24, 36, 39]. New architectures are also proposed, such as the Spectral Neural Operator (SNO, [7]), the Convolutional Neural Operator (CNO, [32]), and transformer-based operator learning [5, 11].

The key advantage of neural operators lies in their ability to generalize across different discretizations of the domains, making them highly versatile for various applications, such as weather forecasting [28], seismic waves [17] or computational neuroscience and cardiology [29]. By leveraging techniques from functional analysis, harmonic analysis, and deep learning, neural operators can be trained on data generated from traditional numerical PDE approximation or with physical measures and subsequently used to predict solutions for new inputs. Moreover, it is theoretically demonstrated (see [15, 26] for DeepONets, [13] for FNOs, [32] for CNOs and [16] for NOs in general), that neural operators can approximate the solution operators for many families of PDEs, enabling rapid and accurate predictions for new instances of the problem. This capability is particularly valuable in scenarios where traditional numerical methods are computationally expensive or infeasible, such as for high-dimensional problems, many query problems (e.g., uncertainty quantification or inverse problems) and real-time simulations. The versatility of neural operators has led to their adoption in a wide range of scientific applications and engineering domains.

Despite their impressive capabilities, training neural operators presents several challenges. One of the primary challenges is the selection of appropriate hyperparameters, which play a critical role in determining the performance and generalization ability of the model. Hyperparameters include architectural choices (e.g., the number of layers, the type of activation functions, and the size of the hidden layers), optimization parameters (e.g., learning rate, batch size, and optimizer type), and regularization parameters (e.g., learning rate scheduler and weight decay). Because the hyperparameter space is often large and complex, manual hyperparameter tuning is a labour-intensive and error-prone process that often requires domain expertise, and banal search spaces, like grid search, are too computationally expensive. Automated hyperparameter tuning is a fundamental component of modern machine learning workflows, especially for complex models such as neural operators. The process involves systematically searching the hyperparameter space to identify configurations that maximize model performance, as measured by a predefined loss function. Automated hyperparameter tuning provides several key benefits:

- **Improved Model Performance:** The performance of machine learning models, especially neural operators, is highly sensitive to the choice of hyperparameters. Automated tuning identifies configurations that maximize model performance on validation data, leading to better generalization and robustness. This is particularly important for neural operators, where the relationship between hyperparameters and performance is non-intuitive and problem-dependent.
- **Efficiency and Automation:** Automated tuning algorithms systematically explore the hyperparameter space, reducing the need for manual intervention. Manual hyperparam-

eter tuning is time-consuming and labour-intensive, requiring extensive trial and error and some domain knowledge, this automation saves time and allows researchers to focus on higher-level tasks, such as model design and interpretation of results.

- **Adaptability to Different Domains:** Neural operators are used in a wide variety of domains and applications, each with its own unique challenges and requirements. Automated tuning can adapt to these diverse contexts by optimizing hyperparameters for specific tasks, datasets, and performance metrics.
- **Scalable and Parallel:** As models and datasets grow in size and complexity, the hyperparameter optimization cost grows exponentially. Manual tuning becomes impractical for large problems due to the sheer number of possible configurations. Automated tuning methods efficiently handle large and high-dimensional search spaces using parallel computing and concentrating on promising hyperparameter configurations with automatic stopping of the bad trials.
- **Reproducibility:** Reproducibility is a cornerstone of scientific research, ensuring that results can be independently verified and validated. We provide a systematic approach that stores hyperparameter configurations and trained models to ensure reproducible results.
- **Human and Machine Efficiency:** As mentioned above, human intuition is limited in navigating complex hyperparameter spaces, especially for high-dimensional models such as neural operators. We show numerically that by offloading the tuning process to machines, trained models are not affected by local minima due to suboptimal hyperparameter choices.

To address this challenge, we introduce **HyperNOs**, a PyTorch library designed to streamline and automate the process of exploring neural operators, with a special focus on hyperparameter optimization for comprehensive and exhaustive exploration. HyperNOs takes advantage of state-of-the-art optimization algorithms and parallel computing implemented in the Ray-tune library to efficiently explore the hyperparameter space of neural operators. We also implement many useful functionalities for studying neural operators with a user-friendly interface. Furthermore, we provide a set of code examples with different datasets and different neural operator architectures, in particular with Fourier neural operators and convolutional neural operators, to demonstrate the capabilities of HyperNOs to handle real datasets and modern architectures. For each example, we provide the associated optimal hyperparameter configuration found and provide the final trained model for future testing. The library is designed to be easy to use with the provided model and datasets, but also to be extended to use new datasets and custom neural operator architectures.

In Sect. 2 we provide an overview of neural operators in general, and then we resume the definition of FNOs and CNOs architectures, highlighting their similarities and differences. In Sect. 3 we present the design and main features of the HyperNOs library for the study of neural operators, through an example of an application. In Sect. 4 we show how to use the HyperNOs library with more advanced features that are implemented, such as finding hyperparameters with a given and fixed complexity, enabling multiresolution training or multiple datasets training. In Sect. 5 we provide some numerical experiments to demonstrate the capabilities of the HyperNOs library. Finally, in Sect. 6 we provide some conclusions and future developments.

2 Overview on neural operators

Neural operators have emerged as a powerful framework for learning mappings between infinite dimensional function spaces, making them particularly useful at solving partial differential equations (PDEs); indeed, neural operators can capture complex nonlinear dependencies and provide efficient approximations. One of the key advantages of neural operators is their ability to generalize beyond the training data. Once trained, a neural operator can provide solutions to a family of PDEs without the need for retraining, significantly reducing the computational cost compared to traditional solvers. This property is particularly beneficial in scenarios where real-time predictions or rapid exploration of parameter spaces are required.

2.1 Problem formulation

We consider a generic family of partial differential equations (PDEs) defined in a bounded domain $D \subset \mathbb{R}^d$ with boundary ∂D . The PDEs are of the form:

$$\begin{cases} (\mathcal{L}_a u)(x) = f(x), & x \in D, \\ (\mathcal{B}u)(x) = g(x), & x \in \partial D, \end{cases}$$

where:

- $\mathcal{L}_a$ is a differential operator parameterized by $a \in \mathcal{A}(D, \mathbb{R}^{d_a})$,
- $u : D \rightarrow \mathbb{R}^{d_u}$ is the solution to the PDE,
- $f : D \rightarrow \mathbb{R}^{d_u}$ is a forcing term or source function.
- $\mathcal{B}$ is possibly a differential operator that with $g : \partial D \rightarrow \mathbb{R}^{d_u}$ define the boundary condition.

The parameter a belongs to the Banach space $\mathcal{A}(D, \mathbb{R}^{d_a})$, the solution u lives in the Banach space $\mathcal{U}(D, \mathbb{R}^{d_u})$ and the forcing term f belongs to the dual space $\mathcal{U}^*(D, \mathbb{R}^{d_u})$, which consists of functionals acting on $\mathcal{U}(D, \mathbb{R}^{d_u})$. The operator $\mathcal{L}_a$ maps the parameter a with a possibly non-linear differential operator $\mathcal{L}_a : \mathcal{U}(D, \mathbb{R}^{d_u}) \rightarrow \mathcal{U}^*(D, \mathbb{R}^{d_u})$. The solution operator $\mathcal{G}^\dagger := \mathcal{L}_a^{-1} f$ is defined as the mapping from the parameter a to the solution u , i.e., $\mathcal{G}^\dagger : \mathcal{A}(D, \mathbb{R}^{d_a}) \rightarrow \mathcal{U}(D, \mathbb{R}^{d_u})$. This operator is central to our problem, as it encapsulates the relationship between the parameter a and the solution u . Our goal is to approximate $\mathcal{G}^\dagger$ using data-driven methods.

In the previous example, we considered the case of a PDE with a single and fixed forcing term and boundary conditions, but the framework can be trivially modified to consider the forcing term or the boundary condition as the parameter and take a as a constant. Furthermore, the framework can be extended to consider multiple parameters at the same time, i.e., varying both source terms, boundary conditions and the geometry of the problem [1].

2.2 Data and discretization

To make the problem computationally tractable, we assume that the domain D is discretized into $n_{points} \in \mathbb{N}$ points. Let $\{a^{(i)}, u^{(i)}\}_{i=1}^N$ be a dataset of $N \in \mathbb{N}$ pairs of coefficient functions and solution functions, where $a^{(i)}$ are i.i.d. samples from a probability measure μ supported on $\mathcal{A}$, and $u^{(i)} = \mathcal{G}^\dagger(a^{(i)})$ are the corresponding solutions.

The dataset $\{a^{(i)}, u^{(i)}\}_{i=1}^N$ is used to train a neural operator model, $\mathcal{G}_\theta : \mathcal{A} \rightarrow \mathcal{U}$, that approximates the solution operator $\mathcal{G}^\dagger$. We are interested in controlling the error of the

approximation on average with respect to μ , more precisely we want to approximate the solution of the following problem:

$$\theta^* = \underset{\theta \in \Theta}{\operatorname{argmin}} \mathbb{E}_{a \sim \mu} \operatorname{Loss}(\mathcal{G}^\dagger(a), \mathcal{G}_\theta(a)) \approx \underset{\theta \in \Theta}{\operatorname{argmin}} \frac{1}{N} \sum_{i=1}^N \operatorname{Loss}(u^{(i)}, \mathcal{G}_\theta(a^{(i)})),$$

where Loss is a suitable loss function, and Θ is the set of trainable parameters of the neural operator model. A standard choice for the loss function is the relative L^p error between the true solution and the predicted solution

$$\operatorname{Loss}(u, \mathcal{G}_\theta(a)) = \frac{\|u - \mathcal{G}_\theta(a)\|_{L^p(D)}}{\|u\|_{L^p(D)}} \approx \frac{\left(\sum_{j=1}^{n_{\text{points}}} |u(x_j) - \mathcal{G}_\theta(a)(x_j)|^p\right)^{1/p}}{\left(\sum_{j=1}^{n_{\text{points}}} |u(x_j)|^p\right)^{1/p}}, \quad p \geq 1.$$

The final discretized problem is to approximate the parameters of the model that satisfy the following minimization problem

$$\theta^* \approx \underset{\theta \in \Theta}{\operatorname{argmin}} \frac{1}{N} \sum_{i=1}^N \frac{\left(\sum_{j=1}^{n_{\text{points}}} |u^{(i)}(x_j) - \mathcal{G}_\theta(a^{(i)})(x_j)|^p\right)^{1/p}}{\left(\sum_{j=1}^{n_{\text{points}}} |u^{(i)}(x_j)|^p\right)^{1/p}}.$$

We emphasize that we discretize the problem in order to compute it numerically, but a key feature of neural operators is that they can learn the solution operator in function spaces, so the model definition is independent of the resolution of the discretization of the domain. We mention that other loss functions can be used, e.g., the H^p relative error. For our exposition, and for the neural operator models that we will consider in the coming sections, the previous discretization is natural, but we note that other types of discretization, with appropriate neural operator architecture, can be developed, for example, using truncated basis coefficients [7].

2.3 Multi-layer perceptron

We briefly define **Multi-Layer Perceptron (MLP)** as a class of feedforward neural networks consisting of multiple layers of neurons, where each neuron applies a nonlinear activation function to its inputs. Mathematically, an MLP with L layers can be defined as follows. Let $x \in \mathbb{R}^{n_0}$ be the input vector, and let $h^{(l)} \in \mathbb{R}^{n_l}$ denote the output of the l -th layer for $l = 1, 2, \dots, L+1$. The MLP is defined by the following recursive relation:

$$\begin{cases} h^{(l)} = \sigma(W^{(l)}h^{(l-1)} + b^{(l)}), & l = 1, 2, \dots, L, \\ h^{(L+1)} = W^{(L+1)}h^{(L)} + b^{(L+1)}, \end{cases}$$

where $h^{(0)} := x$ is the input to the first layer, $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$, for $l = 1, \dots, L+1$, is the weight matrix of the l -th layer, $b^{(l)} \in \mathbb{R}^{n_l}$, for $l = 1, \dots, L+1$, is the bias vector of the l -th layer, $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ is a nonlinear activation function applied element-wise and $h^{(L+1)} \in \mathbb{R}^{n_{L+1}}$, is the final output vector. The Fig. 1 illustrates a visual representation of an MLP with $L = 2$ hidden layers.

2.4 Neural operators

In this section, we provide a general overview of neural operators. In particular, we aim to provide a general definition of neural operators, following [14], highlighting the main

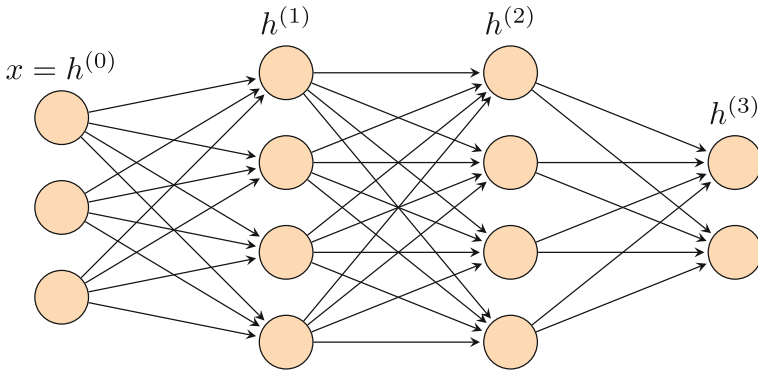


Fig. 1 Representation of a MLP with $L = 2$ hidden layers

properties that a neural operator needs to satisfy. We then specify the NO definition for the FNO and CNO models.

We define a neural operator as a composition of other operators as follows:

$$\mathcal{G}_\theta : \mathcal{A}(D, \mathbb{R}^{d_a}) \rightarrow \mathcal{U}(D, \mathbb{R}^{d_u}), \quad \text{with } D \subset \mathbb{R}^d, \quad \mathcal{G}_\theta := \mathcal{Q} \circ \mathcal{L}_L \circ \dots \circ \mathcal{L}_1 \circ \mathcal{P}.$$

1. **Lifting operator:** The lifting operator is the first step in the neural operator architecture. It maps the input function from its original function space (d_a) to a higher-dimensional space (d_v). This is useful because the input function is often low-dimensional (e.g., scalar-valued), while the neural operator requires a higher-dimensional representation to effectively process and extract features. The lifting operator is defined as follows:

$$\mathcal{P} : \mathcal{A}(D, \mathbb{R}^{d_a}) \rightarrow \mathcal{U}(D, \mathbb{R}^{d_v}), \quad v_0(x) = \mathcal{P}(a)(x) = \mathcal{P}(a(x)) \in \mathbb{R}^{d_v}.$$

The lifting operator is typically implemented as an MLP (defined above) or a convolution block (defined later in Sect. 2.6) that acts locally on the input function. Generally, it is a shallow network or even a linear layer, respectively, an MLP (or convolution block) with one and zero hidden layers.

2. **Integral operator:** The integral operator is the core component of the neural operator architecture. It captures the global interactions and dependencies in the input function by applying a sequence of linear and non-linear transformations. The integral operator is composed of L layers, each of which is as follows

$$\mathcal{L}_t : \mathcal{U}(D, \mathbb{R}^{d_v}) \rightarrow \mathcal{U}(D, \mathbb{R}^{d_v}), \quad t = 1, \dots, L.$$

The resulting operator is a composition of L layers, and this composition has to be non-linear and non-local in order to approximate the solution operator coming from the PDEs.

3. **Projection operator:** The projection operator is the final step in the neural operator architecture. It maps the high-dimensional internal representation of the network (d_v) back to the output function space (d_u).

$$\mathcal{Q} : \mathcal{U}(D, \mathbb{R}^{d_v}) \rightarrow \mathcal{U}(D, \mathbb{R}^{d_u}), \quad \mathcal{Q}(v_L)(x) = \mathcal{Q}(v_L(x)) = u(x) \in \mathbb{R}^{d_u}.$$

The projection operator acts pointwise and can be non-linear; the implementation is the same for the lifting operator.

2.5 Fourier neural operators

Fourier neural operators are a class of neural operators that use the Fourier transform to efficiently parameterize the integral operator defined in the general neural operator framework. As the Fourier transform is a global operator, each layer of the neural operator is non-local and is defined as non-linear by composition with a non-linear activation function. By operating in the frequency domain, FNOs can capture global dependencies in the input functions, making them highly effective for problems with periodic boundary conditions or smooth solutions.

More precisely the integral operator for FNOs is defined as follows:

$$v_{t+1}(x) = \mathcal{L}_t(v_t)(x) := \sigma \left(W_t v_t(x) + b_t + (\mathcal{K}_t(a, \theta_t) v_t)(x) \right),$$

$$W_t \in \mathbb{R}^{d_v \times d_v}, \quad b_t \in \mathbb{R}^{d_v}, \quad t = 0, \dots, L-1, \quad (1)$$

$\theta_t \in \Theta_t \subset \Theta$ is a subset of all the trainable parameters, σ is a non-linear activation function. Moreover, $\mathcal{K}_t(a, \theta_t)$ is called the **integral kernel operator**. The first two addends in (1) are a pointwise and linear transformation, whereas the integral kernel operator is a linear but non-local operator. Moreover, for the definition of the Fourier neural operator, we make the following assumptions

$$(\mathcal{K}_t(a, \theta_t) v_t)(x) = (\mathcal{K}_t(\theta_t) v_t)(x) = \int_D \kappa_{t, \theta_t}(x, y) v_t(y) dy = \int_D \kappa_{t, \theta_t}(x - y) v_t(y) dy$$

$$= (\kappa_{t, \theta_t} * v_t)(x),$$

where we assume $D = \mathbb{T}^d$ that is the d -dimensional torus, $\kappa_{t, \theta_t}(z) \in \mathbb{R}^{d_v \times d_v}$, $z \in D$, is a learnable function usually called kernel function, and $*$ denotes the convolution of functions. In FNOs, the kernel is parameterized in the Fourier domain, allowing for efficient computation and global feature extraction; indeed, using the convolution theorem, we have

$$(\kappa_{t, \theta_t} * v_t)(x) = \mathcal{F}^{-1} \left(\mathcal{F}(\kappa_{t, \theta_t})(k) \cdot \mathcal{F}(v_t)(k) \right) (x),$$

where $\mathcal{F}$, $\mathcal{F}^{-1}$ are respectively the Fourier transform and the inverse Fourier transform. Furthermore $\mathcal{F}(\kappa_{t, \theta_t})(k) \in \mathbb{C}^{d_v \times d_v}$, $\mathcal{F}(v_t)(k) \in \mathbb{C}^{d_v}$ for every $k \in \mathbb{Z}^d$ and the multiplication dot represents the standard matrix-vector multiplication. Parameterizing $\mathcal{F}(\kappa_{t, \theta_t})(k)$ with learnable parameters $R_{\theta_t}(k) \in \mathbb{C}^{d_v \times d_v}$, $\forall k \in \mathbb{Z}^d$, we obtain

$$(\mathcal{K}_t(\theta_t) v_t)(x) = \mathcal{F}^{-1} \left(R_{\theta_t}(k) \cdot \mathcal{F}(v_t)(k) \right) (x). \quad (2)$$

Since the function κ_{t, θ_t} is a real-valued function that is parameterized by $R_{\theta_t}(k)$, $k \in \mathbb{Z}^d$, we impose that the parameters satisfy the Hermitian property, i.e., $R_{\theta_t}(k) = \overline{R_{\theta_t}(-k)}$, $\forall k \in \mathbb{Z}^d$.

To simplify the understanding of the FNO we provide Fig. 2 for a visualization of the model architecture for a video representation, see [9].

As a final remark, we mention that in practice, some slight modifications of (1) are considered. In particular, we introduce the following three notations:

$$v_{t+1}^{Classic} = \mathcal{L}_t^{Classic}(v_t) := \sigma \left(W_t v_t + b_t + \mathcal{K}_t(\theta_t) v_t \right),$$

$$v_{t+1}^{MLP} = \mathcal{L}_t^{MLP}(v_t) := \sigma \left(W_t v_t + b_t + MLP(\mathcal{K}_t(\theta_t) v_t) \right),$$

$$v_{t+1}^{Res} = \mathcal{L}_t^{Res}(v_t) := v_t + \sigma \left(W_t v_t + b_t + \mathcal{K}_t(\theta_t) v_t \right).$$

Where the first one is exactly (1) with a different notation, the second one adds an MLP acting pointwise after the integral kernel operator and the third one is a residual-like modification. It

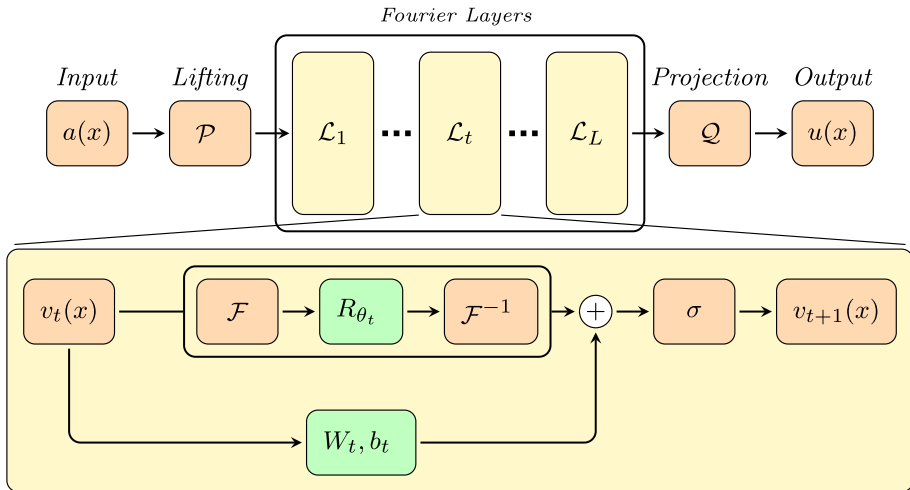


Fig. 2 Visual representation of a Fourier Neural Operator

is numerically shown that employing skip connections to Fourier layers enables the training of deeper FNOs [36].

2.6 Convolutional neural operators

Convolutional neural operators extend the concept of Convolutional Neural Networks (CNNs), in particular U-Nets [34], to the setting of neural operators. As shown in [2], a Representative Equivalent Neural Operator (RENO) has to satisfy the Continuous-Discrete Equivalence (CDE) in order to learn the underlying operator and not only a discrete representation of it. This feature is crucial for neural operators, to ensure stable model performance across different discretization resolutions. The independence of the architecture from the discretization resolution (also called the resolution invariance property) is necessary but not sufficient to be RENO. Indeed, CNOs are carefully designed to satisfy the resolution invariant and the CDE condition. CNOs are defined on the space of bandlimited functions, defined as

$$\mathcal{B}_\omega(D, \mathbb{R}^{d_v}) = \left\{ f \in L^2(D, \mathbb{R}^{d_v}) : \text{supp} \mathcal{F}(f) \subseteq [-\omega, \omega]^2 \right\},$$

where $\omega > 0$, $\mathcal{F}(f)$ denotes the Fourier transform of the function f and throughout this section we assume D a rectangular domain. From the definition of the convolutional neural operator, it follows that CNO maps from the space of bandlimited functions to the space of bandlimited functions, i.e.,

$$\mathcal{G}_\theta : \mathcal{B}_\omega(D, \mathbb{R}^{d_a}) \rightarrow \mathcal{B}_\omega(D, \mathbb{R}^{d_u}).$$

With the previous settings, we define the lifting and projection operator as a convolutional block (defined later in the section) and the integral operator for CNOs as follows:

$$v_{t+1} = \mathcal{L}_t(v_t) := \mathcal{P}_t \circ \Sigma_t \circ \mathcal{K}_t(v_t), \quad t = 0, \dots, L-1. \quad (3)$$

with $v_t, v_{t+1} \in \mathcal{B}_\omega(D, \mathbb{R}^{d_v})$. The lifted function (v_0) is processed through the composition of a series of mappings between functions (layers), with each layer consisting of three

elementary operators, i.e., $\mathcal{P}_t$ is either the *upsampling* or *downsampling* operator, $\mathcal{K}_t$ is the convolution operator and $\mathcal{S}_t$ is the activation operator. In the remainder of this section, we are going to describe these three elementary operators. Furthermore, to construct the typical U-Net architecture, in addition to the operators mentioned above, we define the residual operator and the invariant operator.

To summarize, the input function is lifted through the lifting operator, then the lifted function passes through a sequence of encoders (where the resolution is reduced and the channel size is increased), then passes through a sequence of decoders (where the resolution is increased and the channel size is decreased) and finally the output function is obtained through the projection operator. More precisely, let r be the initial resolution and c be the initial number of channels relative to the initial data. We define a hyperparameter called the channel multiplier (*chan_mul*) such that the CNO architecture is constructed to have, at the i -th layer for $i = 1, \dots, L$, a resolution equal to $r/2^{i-1}$ and a number of channels equal to c in the input layer and $2^{i-1} \cdot c \cdot \text{chan_mul}$ for $i \geq 2$. For example, in Fig. 3 we have $r = 64$, $c = 1$ and $\text{chan_mul} = 16$. Furthermore, encoders and decoders are connected through skip connections using the residual and invariant operators block, this is needed for the transfer of high-frequency content to the output before filtering it out with the downsampling block. For a visual representation of the CNO architecture refer to Fig. 3. The figure illustrates the flow of the input function through the lifting operator, encoder-decoder sequence, and projection operator, as well as the skip connections and residual blocks that connect the encoders and decoders.

Convolution operator

The convolution operator (or convolution block) $\mathcal{K}_{\omega, \theta} : \mathcal{B}_{\omega}(D, \mathbb{R}^{d_v}) \rightarrow \mathcal{B}_{\omega}(D, \mathbb{R}^{d_v})$ is the analogue of the kernel integral operator for the FNOs, where the suffix ω indicates that the operator maps functions of bandwidth ω , and the suffix θ indicates that the operator is parameterized by some learnable parameters. We make the following assumptions

$$\mathcal{K}_{\omega, \theta} f(x) = (f * \kappa_{\omega, \theta})(x) = \int_D \kappa_{\omega, \theta}(x) f(x - y) dy, \quad f \in \mathcal{B}_{\omega}(D, \mathbb{R}^{d_v}), \quad \forall x \in D.$$

In the previous formula $\kappa_{\omega, \theta}(x)$, usually called kernel function, and $f(x - y)$ belongs to $\in \mathbb{R}^{d_v \times d_v}$ and $\in \mathbb{R}^{d_v}$ respectively, for every $x, y \in D$. Therefore, discretizing and truncating the integral, we obtain

$$\begin{aligned} \mathcal{K}_{\omega, \theta} f(x) &= \int_D \kappa_{\omega, \theta}(y) f(x - y) dy \approx \sum_{i, j=1}^k \kappa_{\omega, \theta}(y_{ij}) f(x - y_{ij}) \\ &= \sum_{i, j=1}^k \kappa_{ij} \cdot f(x - y_{ij}), \quad \forall x \in D. \end{aligned} \quad (4)$$

In this context, k is usually called the kernel size of the convolution and d_v is called the channel dimension.

Following the definition of the convolution operator in (4), the kernel function is parameterized in physical space instead of Fourier space as in (2). So our parameterization is of a *local* nature, but we recover non-locality by composing several of these layers.

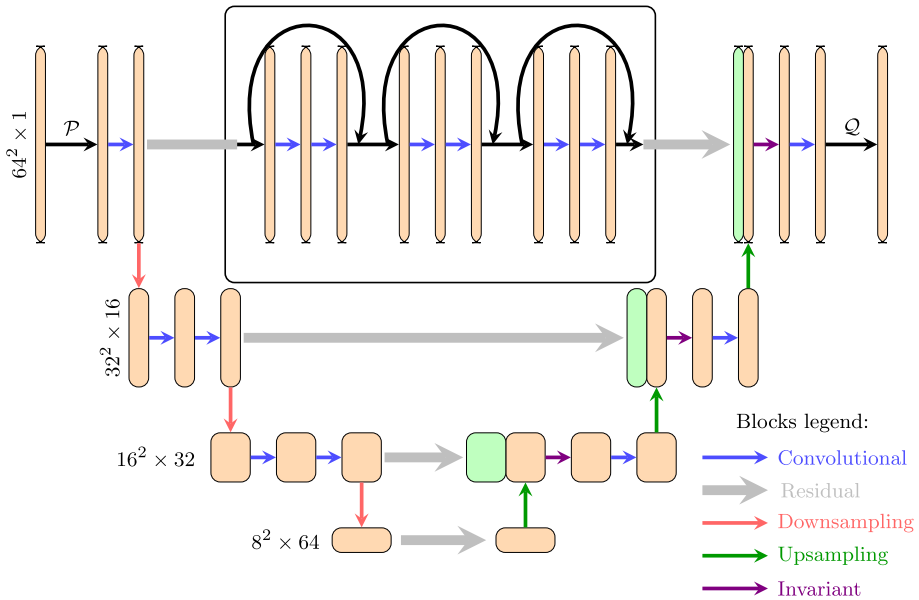


Fig. 3 Visual representation of a CNO architecture with a channel multiplier of 16 and an input with initial resolution 64 and one channel. Each rectangular block represents a set of discretized functions, with its size annotated as $height \times width \times channels$; where $channels$ represent the number of functions and $height = width$ represents the discretization's resolution. Rectangular colours indicate their role in the architecture: orange channels correspond to the standard U-net architecture [34]; grey channels are the ones involved in the residual block (grey arrow); green channels are the output of the residual block. According to the legend: red arrows show downsampling steps that reduce resolution while increasing channels, whereas green arrows show upsampling steps; blue arrows denote the convolutional blocks; gray arrows indicate residual blocks, purple arrows mark the invariant blocks apply on the concatenation of the channels coming from the upsampling block and the residual block; finally black arrows represents skip connections, allowing information to bypass deeper layers and aiding training stability

Upsampling and downsampling operators

For some $\bar{\omega} > \omega$, we can upsample a function $f \in \mathcal{B}_{\omega}(D, \mathbb{R})$ to the higher band $\mathcal{B}_{\bar{\omega}}(D, \mathbb{R})$ by setting,

$$\mathcal{U}_{\omega, \bar{\omega}} : \mathcal{B}_{\omega}(D, \mathbb{R}) \rightarrow \mathcal{B}_{\bar{\omega}}(D, \mathbb{R}), \quad \mathcal{U}_{\omega, \bar{\omega}} f(x) = f(x), \quad \forall x \in D.$$

This is trivial since $f \in \mathcal{B}_{\omega}(D, \mathbb{R}) \subset \mathcal{B}_{\bar{\omega}}(D, \mathbb{R})$, so upsampling does not change the function itself; it reinterprets f as belonging to the higher bandlimited space.

On the other hand, for some $\underline{\omega} < \omega$, we can downsample a function $f \in \mathcal{B}_{\omega}(D, \mathbb{R})$ to the lower band $\mathcal{B}_{\underline{\omega}}(D, \mathbb{R})$ by setting $\mathcal{D}_{\omega, \underline{\omega}} : \mathcal{B}_{\omega}(D, \mathbb{R}) \rightarrow \mathcal{B}_{\underline{\omega}}(D, \mathbb{R})$, defined by

$$\mathcal{D}_{\omega, \underline{\omega}} f(x) = \left(\frac{\omega}{\underline{\omega}}\right)^2 (h_{\underline{\omega}} * f)(x) = \left(\frac{\omega}{\underline{\omega}}\right)^2 \int_D h_{\underline{\omega}}(x - y) f(y) dy, \quad \forall x \in D,$$

where $\left(\frac{\omega}{\underline{\omega}}\right)^2$ is a proper normalization factor and $h_{\underline{\omega}}$ is the so-called interpolation sinc filter, that is a low-pass filter of f that removes frequencies higher than $\underline{\omega}$, defined as:

$$h_{\omega}(x) = \prod_{i=1}^d \text{sinc}(2\omega x_i), \quad x \in \mathbb{R}^d,$$

where $\text{sinc}(x) = \sin(\pi x)/(\pi x)$ is the sinc function.

For function that are vector-valued $f \in \mathcal{B}_\omega(D, \mathbb{R}^d)$, for some $d \in \mathbb{N}$ and $d > 1$, we define the upsampling operator $\mathcal{U}_{\omega, \bar{\omega}} : \mathcal{B}_\omega(D, \mathbb{R}^d) \rightarrow \mathcal{B}_{\bar{\omega}}(D, \mathbb{R}^d)$ and the downsampling operator $\mathcal{D}_{\omega, \bar{\omega}} : \mathcal{B}_{\bar{\omega}}(D, \mathbb{R}^d) \rightarrow \mathcal{B}_\omega(D, \mathbb{R}^d)$ componentwise, i.e. we apply the scalar definition to each dimension independently.

Activation operator

Naively, it is common in machine learning to apply the activation function pointwise to any function. However, it has been shown (see [2]) that such an application can generate features that do not respect the bandlimits of the underlying function space, leading to aliasing errors. To avoid aliasing errors and satisfy the CDE condition, we first upsample the input function $f \in \mathcal{B}_\omega(D, \mathbb{R})$ to a higher bandlimit $\bar{\omega} > \omega$, then apply the activation, assuming that $\bar{\omega}$ is large enough so that $\sigma(\mathcal{B}_\omega(D, \mathbb{R})) \subset \mathcal{B}_{\bar{\omega}}(D, \mathbb{R})$, and finally downsample the result back to the original band limit ω . So we define the activation operator, or convolutional block, as

$$\Sigma_\omega : \mathcal{B}_\omega(D, \mathbb{R}) \rightarrow \mathcal{B}_\omega(D, \mathbb{R}), \quad \Sigma_\omega f(x) = (\mathcal{D}_{\bar{\omega}, \omega} \circ \sigma \circ \mathcal{U}_{\omega, \bar{\omega}} f)(x), \quad \forall x \in D.$$

As with the upsampling and downsampling operators, we define the activation operator for the vector-valued function $\mathcal{B}_\omega(D, \mathbb{R}^d)$ componentwise.

Residual and invariant operators

The residual and invariant operators are defined in relation to the previously defined operators. The residual (or skip-connection) operator connects the encoder and decoder blocks with the same spatial resolution, and this is useful for transferring high frequency content to the output before it is filtered out by the downsampling block. In particular, the output of the residual block is concatenated with the corresponding decoder block. The residual operator is a concatenation of the convolutional operator and the activation operator with skip connections; the architecture emulates the classical Residual Neural Network, in particular

$$\begin{aligned} \mathcal{R}_{\omega, \theta} : \mathcal{B}_\omega(D, \mathbb{R}^{d_v}) &\rightarrow \mathcal{B}_\omega(D, \mathbb{R}^{d_v}), \\ \mathcal{R}_{\omega, \theta}(v) &= v + \mathcal{K}_{\omega, \theta} \circ \Sigma_\omega \circ \mathcal{K}_{\omega, \theta}(v), \quad \forall v \in \mathcal{B}_\omega(D, \mathbb{R}^{d_v}). \end{aligned}$$

Where we indicate with $D \subset \mathbb{R}^d$ the domain and $d_v \in \mathbb{N}$ the number of channels of the input function.

At every level of the architecture, after the concatenation between the output of the residual block and the output of the decoder block, we apply the invariant block that is defined as

$$\mathcal{I}_{\omega, \theta} : \mathcal{B}_\omega(D, \mathbb{R}^{d_v}) \rightarrow \mathcal{B}_\omega(D, \mathbb{R}^{d_v}), \quad \mathcal{I}_{\omega, \theta}(v) = \Sigma_\omega \circ \mathcal{K}_{\omega, \theta}(v), \quad \forall v \in \mathcal{B}_\omega(D, \mathbb{R}^{d_v}).$$

2.7 Comparison of FNOs and CNOs

While both FNOs and CNOs are powerful architectures for learning mappings between function spaces, they differ in their mathematical foundations:

- **Global vs. local discretization:** FNOs discretize the kernel function in Fourier space, employing a global discretization at each layer. Conversely, CNOs discretize the kernel function in physical space using a local discretization at each layer.

- **Computational efficiency:** FNOs utilize the Fast Fourier Transform (FFT) for efficient computation, while CNOs rely on generalized convolution operations, which can be computationally expensive for large domains.
- **Resolution independence:** Both FNOs and CNOs are resolution independent, meaning that they can process input functions of different sizes without requiring retraining. It is noteworthy that only CNOs possess the property of being representative equivalent neural operators and satisfying the CDE condition, which is a fundamental theoretical property for neural operators.
- **Translation equivariance:** CNOs are characterized by their inheritance of the translation equivalence property from the convolution operator, a property that is absent in FNOs. The property under consideration is shown to result in enhanced generalization capabilities of the model when the problem is expected to be invariant under translations of the input. This assertion is supported by numerical evidence presented in the study CNO23raonic.

Fourier neural operators and convolutional neural operators represent two powerful approaches to learning mappings between function spaces. Collectively, these architectures furnish a versatile toolkit for addressing complex problems in scientific computing, engineering, and related domains. The subsequent section will discuss the development of automated hyperparameter optimization routines, which enhance the practicality and scalability of these methods. These developments pave the way for their widespread adoption in real-world applications.

3 Explanation of HyperNOs library design

This section presents a concise exposition of the HyperNOs library, with a particular emphasis on the pivotal components and features that render it a valuable instrument for the study of neural operators. The library's design is predicated on a modular architecture, a feature that enables users to define custom search spaces, models, datasets, and loss functions. This attribute renders the library highly flexible and extensible, further complemented by a user-friendly interface. In this section, a model problem and an explicit code example will be used to elucidate the library.

3.1 Model problem and dataset

As a model problem, we consider the Darcy problem, which is a well-known example of a partial differential equation that arises in many applications and has often been used as a starting point for experimentally testing the behaviour of neural operators. We consider the Darcy flow equation in two dimensions, subject to homogeneous Dirichlet boundary conditions, the second-order elliptic equation, given by the following equation

$$\begin{cases} -\nabla(a \cdot \nabla u) = f, & \text{in } D \\ u = 0, & \text{on } \partial D \\ f = 1, & \text{in } D \end{cases} \quad (5)$$

where $D = (0, 1)^2$ is the unit square and we fix the source term equal to 1. In this setting $\mathcal{A} = L^\infty(D, \mathbb{R}^+)$, $\mathcal{U} = H_0^1(D, \mathbb{R})$. We consider the weak form of the solution operator to be the form

$$\mathcal{G} : L^\infty(D, \mathbb{R}^+) \rightarrow H_0^1(D, \mathbb{R}), \quad \mathcal{G} : a \mapsto u.$$

As a dataset, we consider the same created in [32, 33], where the input is the diffusion coefficient $a \sim \psi \# \mu$, with μ being a Gaussian process with zero means and squared exponential kernel

$$k(x, y) = \sigma^2 \exp\left(-\frac{|x - y|^2}{l^2}\right), \quad \sigma^2 = 0.1,$$

with $l = 0.1$ for the in-distribution testing and $l = 0.05$ for the out-of-distribution testing. The mapping $\psi : \mathbb{R} \rightarrow \mathbb{R}$ takes the value 12 on the positive part of the real line and 3 on the negative part.

This dataset has been integrated into the library, specifically in the `datasets.py` module, and can be loaded and used to train different neural operators. The dataset can be downloaded by executing the file named `download_data.sh`, which is located in the library. It can then be loaded using the `Darcy` class, which takes as input the number of Fourier features we want to include, the seed number (if the seed is positive, it is fixed at this number for reproducibility, otherwise no seed is fixed), the dimension of the batch size, and the number of training samples. The code responsible for loading the dataset can be found in Code 1 from line 39 to line 44. Within this code snippet, a wrapper function is employed to invoke the `Darcy` class.

Note that the library contains additional datasets that have already been implemented. Specifically, all the functionalities necessary for the datasets from [32, 33]. The remaining functions employed for loading the datasets can be utilized in a manner analogous to the previously outlined approach. Users can implement their own dataset by inheriting from the `torch.utils.data.Dataset` class. The dataset must be structured such that the batch dimension is designated as the primary dimension, and it is further subdivided into three distinct subsets: the training set, which is utilized for the training process; the validation set, which is employed for the hyperparameter optimization process; and the test set, which is designated for final testing.

3.2 Model architecture

The model architecture is defined by the user via a model builder function, which takes a configuration dictionary as input and returns an instance of the neural operator model. The model builder function is passed to the `tune_hyperparameters` function, which is the main function of the library and is responsible for handling the hyperparameter optimization process. The configuration dictionary contains all the necessary hyperparameters that define the neural operator $\mathcal{G}_\theta$. In this particular example, we consider the architecture of the Fourier neural operator implemented in the `FNO.py` module of the library. The implementation of this approach is guided by Sect. 2.5 with a focus on achieving maximum generalizability. Within the library, we have implemented the latest architecture of convolutional neural operators in the `CNO.py` module, in accordance with Sect. 2.6 with the objective of implementing as many features as possible. The user has the capacity to define their own model architecture by implementing their own class of the model and the associated custom model builder function. This function is then passed to `tune_hyperparameters`. The only requirement for the model is that it accept a single input during the forward method; furthermore, the tensors that are provided as inputs to the model must possess the batch dimension as the primary dimension, followed by the space dimension, denoted by d , and concluding with the input dimension, denoted by d_a . The code for the model builder function is displayed in Code 1 from line 30 to line 37.

Code 1 Code for the model problem.**Model problem code**

```

1 import sys
2 sys.path.append("..")
3 from architectures import FNO
4 from datasets import NO_load_data_model
5 from loss_fun import LprelLoss
6 from ray import tune
7 from tune import tune_hyperparameters
8 from utilities import initialize_hyperparameters
9 import torch
10 device = torch.device("cuda" if torch.cuda.is_available()
11                       else "cpu")
12 hyperparams_train, hyperparams_arc =
13     initialize_hyperparameters("FNO", "darcy",
14                               mode="default")
15 config_space = {
16     "learning_rate": tune.quniform(1e-4, 1e-2, 1e-5),
17     "weight_decay": tune.quniform(1e-6, 1e-3, 1e-6),
18     "scheduler_gamma": tune.quniform(0.75, 0.99, 0.01),
19     "width": tune.choice([4, 8, 16, 32, 64, 96, 128]),
20     "n_layers": tune.randint(1, 6),
21     "modes": tune.choice([2, 4, 8, 12, 16, 20, 24]),
22     "fun_act": tune.choice(["tanh", "relu", "gelu"]),
23     "padding": tune.randint(0, 16) }
24 fixed_params = {**hyperparams_train, **hyperparams_arc}
25 parameters_to_tune = config_space.keys()
26 for param in parameters_to_tune:
27     fixed_params.pop(param, None)
28 config_space.update(fixed_params)
29
30 model_builder = lambda config: FNO(
31     config["problem_dim"], config["in_dim"],
32     config["width"], config["out_dim"],
33     config["n_layers"], config["modes"],
34     config["fun_act"], config["weights_norm"],
35     config["fno_arc"], config["RNN"],
36     config["fft_norm"], config["padding"],
37     device, None, config["retrain"] )
38
39 dataset_builder = lambda config: NO_load_data_model(
40     which_example="darcy",
41     no_architecture={"FourierF": config["FourierF"],
42                     "retrain": config["retrain"] },
43     batch_size=config["batch_size"],
44     training_samples=config["training_samples"] )
45
46 loss_fn = LprelLoss(1)
47
48 tune_hyperparameters(config_space, model_builder,
49                     dataset_builder, loss_fn, num_samples=100,
50                     runs_per_cpu=8.0, runs_per_gpu=0.5)

```

3.3 Loss function

The loss function is defined by the user via a loss function object, which is passed to the `tune_hyperparameters` function. The loss function object should implement the `__call__` method, which takes as input the model output and the target values, in that order, and returns the sum of the loss value along the batch dimension. In this example, we consider the L^1 relative loss function discussed in 2.2 and implemented in the `loss_fun.py` module of the library. The code for the loss function object is shown in Code 1 on line 46.

The user can find already implemented the $L^p(\Omega)$, $\forall p \in [1, +\infty)$ and for all the domains Ω , and $H^p(D)$, $\forall p \in [1, +\infty)$ for rectangular domains D . The user can define their own loss function by implementing a custom loss function object that satisfies the requirements described above.

3.4 Hyperparameter optimization

The last necessary step is to define the hyperparameter search space and call the `tune_hyperparameters` function, which handles the hyperparameter optimization process. The search space is defined by a configuration dictionary, where each key corresponds to a hyperparameter and the value is a tune search space object. The user can define the search space using the `tune` module from the Ray library [27], which provides a wide range of search space objects, including `choice`, `randint`, `uniform`, `loguniform` and `randn`. The configuration space must contain all the hyperparameters that define the neural operator $\mathcal{G}_\theta$, the data set, the loss function, and the hyperparameters for the optimization process. The code for this example is shown in Code 1 from line 14 to line 22. We have fixed some reasonable hyperparameters, such as the number of examples in the train set and the validation set, the number of epochs, the batch size, and many others. We also decide how to vary the more interesting hyperparameters, such as the number of layers, the width of the network, the number of modes, the activation function, the architecture of the network, the learning rate, the weight decay, and so on. This search space can be modified by the user to find the best choice of hyperparameters for an architecture or to study the dependence of the architecture's performance on a few hyperparameters.

After defining the search space, the dataset builder, the model builder and the loss function, we can call the `tune_hyperparameters` function, which performs the hyperparameter optimization process using the Ray-tune library, see line 48 of Code 1. The function has other optional arguments that can be used to customize the optimization process. The number of sample parameters determines how many trials to run to find the best possible hyperparameter. We can decide how many CPU cores and GPUs to use in each trial of the optimization process, and the number of trials to run in parallel, which depends on the available hardware, and this is automatically calculated by the library. In our example Code 1, we allocate half of each GPU to each trial; in other words, the library runs two trials in parallel for each GPU present on the machine. The optimization process is based on the HyperOpt Python library, which mainly uses the Tree-structured Parzen Estimator (TPE) approach and the adaptive TPE, see [3]. We adopt the ASHA algorithm to terminate underperforming trials to speed up the overall process, see [19]. Finally, the library automatically saves checkpoints of the trials, which can be resumed at any time, and provides TensorBoard support for better visualization of the results.

3.5 Final training

After finding the best hyperparameters for the model, the user can train the model using the best hyperparameters found in the optimization process. The user can use the supplied `train_fixed_model` function, which takes as input the chosen hyperparameters, the model builder, the dataset builder and the loss function, as before. We also need to provide a name to identify the test, a function to plot the input and a function to plot the output. The function trains the model on the test set and saves the trained model with the chosen name. During training, the loss function values are stored together with the outputs and the error of the model using Tensorboard for better visualization of the training process.

Practically, the code for the final testing is similar to the one presented in the Code 1, the only difference being that we don't have the config space since every hyperparameter is fixed. The dataset builder, model builder and loss function are the same as before, and we call the function `train_fixed_model` as shown in Code 2.

Code 2 Code for the final training.

Final training

```

1 train_fixed_model (
2     default_hyper_params, # fixed hyperparameters
3     model_builder, # model builder function, defined as before
4     dataset_builder, # dataset builder function, defined as
      before
5     loss_fn, # loss function object, defined as before
6     experiment_name, # name of the experiment
7     get_plot_function("darcy", "input"), # function to plot
      the input
8     get_plot_function("darcy", "output"), # function to plot
      the output
9 )

```

4 Advanced features with examples

In this section, we explain many different modifications of the basic Code 1 that can be used to solve more complex and interesting problems involving neural operators. The features presented are completely independent, so they can be combined with each other.

4.1 Default hyperparameters loading

Within the library, we provide the hyperparameters that have been identified as optimal for a variety of datasets. These hyperparameters have been carefully tuned through extensive experimentation, as detailed in Sect. 5. Making these configurations readily available ensures reproducibility, detailed debugging, efficiency and scalability in experiments. This approach allows researchers to quickly replicate results, analyse model behaviour, and build on existing work without redundant hyperparameter tuning, focusing on higher-level tasks such as model interpretation and application. For readability reasons, the hyperparameters are stored in JSON files and divided into two sub-dictionaries: those needed for the training

process and those needed for the architecture definition. This step ensures that all necessary configurations are available in a consistent format, simplifying the setup process. This structured approach improves the user experience, code readability and promotes best practices in machine learning experimentation. With this in mind, we have implemented some functionality for the user to load and use the desired hyperparameters. An example of how to load and merge these hyperparameters is provided in Code 3.

Code 3 Code for loading the default hyperparameters.

Load default hyperparameters

```

1 # Load the best hyperparameters for the FNO model of the
  Darcy example
2 hyperparams_train, hyperparams_arc =
  initialize_hyperparameters(
3   "FNO", which_example="darcy", mode="best")
4
5 # Alternatively, load the best hyperparameters for the CNO
  model
6 hyperparams_train, hyperparams_arc =
  initialize_hyperparameters(
7   "CNO", which_example="darcy", mode="best")
8
9 # Merge the hyperparameters together
10 fixed_params = {**hyperparams_train, **hyperparams_arc}

```

In addition to the advantages already discussed, the default hyperparameters can also be used within the hyperparameter optimization process itself. When using the `tune_hyperparameters` function, the default hyperparameters can be passed as the initial configuration. This allows the optimization process to start by evaluating the default hyperparameters, which serve as a baseline for comparison with other trials. By using the ASHA algorithm, the optimization process can quickly eliminate poorly performing trials and focus computational resources on more promising configurations. This significantly speeds up the overall optimization process by allowing the algorithm to prioritize trials that are likely to yield better results.

This approach allows users to focus on optimizing only the most critical parameters, while leaving the others at their default values. This eliminates the need to explicitly specify all hyperparameters, reducing the complexity of the configuration space and speeding up the optimization process. For example, if a user is primarily interested in optimizing the learning rate and number of layers, they can leave the remaining hyperparameters (e.g. activation function, batch size, etc.) at their default values.

The modifications required to implement this feature in the configuration space and in the optional argument of the `tune_hyperparameters` function are demonstrated in Code 4. This implementation ensures that default hyperparameters are seamlessly integrated into the optimization workflow, providing a robust and efficient framework for hyperparameter tuning.

4.2 Model with the same number of trainable parameters

In some cases, it may be desirable to compare models with the same number of trainable parameters to evaluate their performance under similar complexity constraints. To facilitate

Code 4 Code for using the default hyperparameters in the hyperparameters optimization process.

Load default hyperparameters for optimization

```

1 hyperparams_train, hyperparams_arc =
  initialize_hyperparameters(
2   "FNO", "darcy", mode=mode_hyperparams)
3
4 config_space = {
5   "fno_arc": tune.choice(["Classic", "Zongyi", "Residual"]),
6   "fun_act": tune.choice(["tanh", "relu", "gelu",
7     "leaky_relu"]),
8   "learning_rate": tune.quniform(1e-4, 1e-2, 1e-5),
9   "modes": tune.choice([2, 4, 8, 12, 16, 20, 24, 28, 32]),
10  "n_layers": tune.randint(1, 6),
11  "padding": tune.randint(0, 16),
12  "scheduler_gamma": tune.quniform(0.75, 0.99, 0.01),
13  "weight_decay": tune.quniform(1e-6, 1e-3, 1e-6),
14  "width": tune.choice([4, 8, 16, 32, 64, 128, 256])}
15
16 # Set all the other parameters to fixed values
17 fixed_params = {**hyperparams_train, **hyperparams_arc}
18 parameters_to_tune = config_space.keys()
19 for param in parameters_to_tune:
20   fixed_params.pop(param, None)
21   config_space.update(fixed_params)
22
23 tune_hyperparameters(
24   config_space, model_builder, dataset_builder,
25   loss_fn, num_samples=50,
26   default_hyper_params=[{**hyperparams_train,
27     **hyperparams_arc}])
28 runs_per_cpu=0.0, runs_per_gpu=1.0)

```

this comparison, we provide a utility function that allows the user to specify the desired number of trainable parameters, and the library will automatically adjust the model architecture accordingly. To achieve this, the function calculates the number of trainable parameters for a default model architecture and then generates only trials with models with a similar number of parameters. Alternatively, the user can directly enter a number of trainable parameters, and the library will generate only architectures with approximately those values of trainable parameters. This feature is useful in practice and allows users to make fair comparisons between models with different hyperparameters, ensuring that performance differences are not solely due to differences in complexity. Moreover, this approach can be interesting for a more intensive study of the hyperparameter space, since the hyperparameter space is limited by this additional constraint.

We calculate the number of trainable parameters of the model given the hyperparameter configuration. For the Fourier neural operator architecture, the number of trainable parameters can be approximated as follows

$$\text{trainable parameters} \approx 2^d \cdot L \cdot d_v^2 \cdot \text{modes}^d, \quad (6)$$

where d is the problem domain dimension, L is the number of layers, d_v is the hidden dimension, and modes is the number of Fourier modes considered. Thus, in order to generate trials with the same number of trainable parameters, we carry out the optimization on the

number of layers and the hidden dimensions; once they are chosen, we compute the number of Fourier modes according to the formula (1) to match as closely as possible the number of parameters of the standard architecture. In addition, we impose that the number of modes be an integer with positive values and satisfy the Nyquist–Shannon sampling theorem [37]. The modification required to implement this feature is simply to remove the *modes* argument from the configuration space, determine the number of modes and then pass it directly to the model builder; for clarity, this is done with helper functions, but the implementation is straightforward. The main changes are shown in Code 5.

Code 5 Code for the model with the same number of trainable parameters.

Same number of trainable parameters for FNOs

```

1 from utilities import initialize_hyperparameters
2 from architectures import FNO, compute_modes,
   count_params_fno
3
4 # Load the baseline model and compute the total of params
5 hyperparams_train, hyperparams_arc =
   initialize_hyperparameters (
6 "FNO", which_example="darcy", mode="default")
7 total_default_params = count_params_fno(fixed_params,
   accurate=False)
8
9 config_space = {...} # Define as before, but without the
   modes
10
11 # Compute the correct number of modes
12 maximum = 32
13 modes = compute_modes(total_default_params, maximum,
   config),
14
15 model_builder = lambda config: FNO(...) # Define as before
   with the computed modes

```

We can do the same with the convolutional neural operator architecture, the number of trainable parameters can be well approximated by the following formula:

trainable parameters

$$\approx k^d \cdot \text{chan_mul}^2 \left(2^{2L-1} M + 2N \left(\frac{1}{4} + \frac{4^{L-1} - 1}{3} \right) + \frac{31}{6} 4^{L-1} - \frac{11}{12} \right), \quad (7)$$

where k is the kernel size of the convolution, chan_mul is the value of the channel multiplier, L is the number of decoding/encoding layers, M is the number of layers in the last residual block, N is the number of layers in all other residual blocks, and d is the problem domain dimension. Obtaining the approximation (7) of the number of parameters is less trivial with respect to FNO, but the modifications needed to implement this feature are minimal, see Code 6. In particular, we optimize k , L , M , N (or a subset of them) and compute chan_mul to generate trials approximating the same number of hyperparameters.

Code 6 Code for the model with the same number of trainable parameters.**Same number of trainable parameters for CNOs**

```

1 from utilities import initialize_hyperparameters
2 from architectures import CNO, compute_channel_multiplier,
  count_params_cno
3
4 # Load the baseline model and compute the total of params
5 hyperparams_train, hyperparams_arc =
  initialize_hyperparameters(
6 "CNO", which_example="darcy", mode="default")
7 total_default_params = count_params_cno(fixed_params,
  accurate=False)
8
9 config_space = {...} # Define as before, but without
  channel multiplier
10
11 # Compute the correct number of modes
12 channel_multiplier=compute_channel_multiplier(total_default_
13 params, config)
14
15 model_builder = lambda config: CNO(...) # Define as before
  with the computed channel multiplier

```

4.3 Multiple datasets concatenation

In many real-world applications, it is common to work with multiple datasets that contain complementary information or represent different aspects of the same problem. In scientific computing, for example, researchers may need to combine data from different experiments, simulations or sources to build a more comprehensive model. To address this need, our library provides support for concatenating multiple data sets into a single training set, allowing users to leverage diverse data sources for model training. This feature allows researchers to incorporate a wide range of data types, formats and sources into their models, improving model robustness, generalization and performance. Since the architecture definition in the neural operator settings does not depend on the discretization resolution of the dataset, the library allows the concatenation of datasets with different resolutions.

The code for concatenating multiple datasets is shown in Code 7. Here the `NO_load_data_model` function is a wrapper that associates the dataset name with the dataset class, and the `concat_datasets` function concatenates the datasets into a single set. The user can specify which datasets to include in the concatenation by providing a list of dataset names, which we provide, or use custom datasets by including them in the `NO_load_data_model` wrapper, or use them directly as in Code 1.

Code 7 Code for concatenating multiple datasets.**Multiple dataset concatenation**

```

1 from datasets import NO_load_data_model, concat_datasets
2 which_example = ["poisson", "darcy"]
3 dataset_builder = lambda config: concat_datasets(
4     *(NO_load_data_model(
5         dataset_name,
6         no_architecture={"FourierF": config["FourierF"],
7           "retrain": config["retrain"] },
8         batch_size=config["batch_size"],
9         training_samples=config["training_samples"] )
10     for dataset_name in which_example))

```

4.4 Multiple resolution datasets training

We have implemented the ability to train the model on the same dataset but loaded with different resolutions. This feature is useful when the user wants to train the model on the same dataset but with different resolutions, for example, to study the effect on model performance or model generality. This flexibility allows users to combine datasets with different levels of detail, granularity and complexity, providing a technique for data augmentation with a more comprehensive and accurate representation of the underlying problem. We have already mentioned all the necessary concepts in Sect. 4.3, with the difference that this time we do not vary the name of the dataset, but the resolution of the discretization. We must ensure that the downsampled resolution divides the original resolution in order to perform a uniform subsampling of the original data. Furthermore, for the FNO architecture, the minimum resolution must be compatible with the maximum number of modes via the Nyquist-Shannon theorem. The code for this function is shown in Code 8.

Code 8 Code for concatenating multiple datasets.**Multiple dataset resolutions**

```

1 from datasets import NO_load_data_model, concat_datasets
2 resolution = [64, 32, 16] # max number of modes is 9
3 dataset_builder = lambda config: concat_datasets(
4     *(NO_load_data_model(
5         which_example="darcy",
6         no_architecture={"FourierF": config["FourierF"],
7           "retrain": config["retrain"] },
8         batch_size=config["batch_size"],
9         training_samples=config["training_samples"],
10        in_size=res)
11     for res in resolution))

```

4.5 Physics informed neural operators

The architecture of the neural operators ensures a continuous representation with respect to the space variables. These features make it natural to introduce a priori physical knowledge into the construction and training of the architecture. We distinguish between weak and strong imposition. The strong imposition of physical knowledge is discussed in the next subsection. Instead, weak imposition consists of introducing physically informed terms into the loss function in order to promote solutions that satisfy certain physical requirements [18, 38]. The loss function is usually modified as follows

$$\mathcal{Loss} = \mathcal{Loss}_{data} + \alpha_{phys} \mathcal{Loss}_{phys},$$

where $\mathcal{Loss}_{data}$ is the data-driven loss function defined in Sect. 2.2, $\mathcal{Loss}_{phys}$ is the loss function derived from the PDE that defines the physics of the problem, and $\alpha_{phys} \in \mathbb{R}$ is a constant scaling that can be considered as another hyperparameter to add at the tuning process. A typical example of a physics-based loss function is the norm of the residual of the PDE (strong or weak formulation) that the neural operators have to approximate. This process is fundamental to Physically Informed Neural Networks (PINNs, [31]) and Physics Informed Neural Operator (PINO, [18]), but it is shown that this additional term can complicate the training procedure [6]. The modification of the loss function to include physically informed loss is in Code 9.

Code 9 Code for defining a physics-informed loss function.

Physics-informed loss function

```

1 from loss_fun import loss_selector
2 loss_fn = LpreLoss(2)
3
4 from loss_fun_with_physics import PoissonResidualFiniteDiff
5 alpha_phys = 0.01
6 loss_phys = PoissonResidualFiniteDiff(alpha=alpha_phys, p
7     = 2) # norm 2 of the residual
8
9 tune_hyperparameters(
10     config_space, model_builder,
11     dataset_builder, loss_fn,
12     default_hyper_params,
13     loss_phys = loss_phys)

```

4.6 Model's wrapper for complex tasks

It may be useful to define post-processing or architectural modifications of the architecture of the neural operator to ensure some properties or physical constraints. This can be achieved with a model wrapper, a wrapper is a class with an initialization method that defines the model and a forward method that takes the input from the network, evaluates the model and then modifies the output before returning it.

For example, we can use wrappers to impose strong physical knowledge. If we know that on a given part of the boundary, the solution must satisfy a fixed Dirichlet boundary condition $u(x) = g(x)$ in $\Gamma_D \subset \partial D$, then we can modify the output of the model to automatically

satisfy the constraint. In particular, we define

$$u(x) = \psi(x)\mathcal{G}_\theta a(x) + g(x),$$

where $\psi(x) = 0$ in Γ_D and $g(x)$ is the Dirichlet boundary condition extended to zero outside of Γ_D .

Another useful case where wrappers are needed is in the definition of the Fourier continuation for FNOs [13]. For non-rectangular domains, we can embed it in a rectangular one and then use FNOs to approximate the padded operator. When computing the loss function, we want to consider only the points on the original (unpadded) domain, and this can be done with a simple mask using wrappers. The code needed to do this is shown in Code 10.

Code 10 Code for defining Fourier continuation.

Fourier continuation wrapper

```

1 # This is implemented inside the library
2 class AirfoilWrapper(nn.Module):
3     def __init__(self, model):
4         super(AirfoilWrapper, self).__init__()
5         self.model = model
6
7     def forward(self, input_batch):
8         output_batch = self.model(input_batch)
9         output_batch[input_batch == 1] = 1
10        return output_batch
11
12    def __getattr__(self, name):
13        if name != "model" and hasattr(self.model, name):
14            return getattr(self.model, name)
15        return super().__getattr__(name)
16
17    def wrap_model_builder(model_builder):
18        def wrapped_builder(config):
19            model = model_builder(config)
20            return AirfoilWrapper(model)
21        return wrapped_builder

```

Fourier continuation wrapper

```

1 model_builder = lambda config: FNO(...) # Define as in
   Code 1
2 model_builder = wrap_model_builder(model_builder)

```

5 Numerical experiments

In this section, we present numerical examples to demonstrate the effectiveness of our library for the study of neural operators. In particular, we have considered the representative PDE benchmarks provided by [32, 33], and for more details on these benchmarks, see that reference. These benchmarks are well suited for testing neural operator architectures, providing a large variety of PDEs with intrinsic computational complexity to ensure the effectiveness of NOs. The benchmarks are briefly described below.

- **Poisson equation** benchmark that represents a linear elliptic PDE, given by

$$\begin{cases} -\Delta u = f & \text{in } D \\ u = 0 & \text{on } \partial D \end{cases}$$

The dataset provides a collection of pairs $(f^{(i)}, u^{(i)})_i$ where $u^{(i)}$ are the evaluation of the solution operator $\mathcal{G} : f \rightarrow u$ from the source term to the solution.

- **Darcy equation** benchmark that represents a second-order non-linear PDE

$$\begin{cases} -\nabla \cdot (a \nabla u) = f & \text{in } D \times (0, T) \\ u = 0 & \text{on } \partial D \end{cases}$$

The dataset provides a collection of pairs $(a^{(i)}, u^{(i)})_i$ where $u^{(i)}$ are the evaluation of the solution operator $\mathcal{G} : a \rightarrow u$ that maps the diffusion coefficient a into the solution u .

- **Navier–Stokes equation** benchmark that represents a non-linear PDE that models the motion of incompressible fluids, given by

$$\begin{cases} \frac{\partial u}{\partial t} + (u \cdot \nabla)u + \nabla p = \nu \Delta u & \text{in } D \times (0, T) \\ \nabla \cdot u = 0 & \text{in } D \times (0, T) \\ u(x, 0) = f(x) & \text{in } D \\ \nu = 4 \cdot 10^{-4} \end{cases}$$

The dataset provides a collection of pairs $(f^{(i)}, u^{(i)})_i$ where $u^{(i)}$ are the evaluation of the solution operator $\mathcal{G} : f \rightarrow u(\cdot, T)$ that maps the initial condition f into the solution at the final time.

- **Wave equation** benchmark that represents a linear hyperbolic PDE, given by

$$\begin{cases} \frac{\partial^2 u}{\partial t^2} - c^2 \Delta u = 0 & \text{in } D \times (0, T) \\ u(x, 0) = f(x) & \text{in } D \\ c = 0.1 \end{cases}$$

The dataset provides a collection of pairs $(f^{(i)}, u^{(i)})_i$ where $u^{(i)}$ are the evaluation of the solution operator $\mathcal{G} : f \rightarrow u(\cdot, T)$ that maps the initial condition f into the solution at the final time $T = 5$.

- **Compressible Euler equation** benchmark that represents the following non-linear PDE

$$\begin{cases} \frac{\partial u}{\partial t} + \nabla \cdot F(u) = 0 \\ u = [\rho, \rho v, E]^T \\ F = [\rho v, \rho v \otimes v + p \mathbf{I}, (E + p)v]^T \end{cases}$$

with freestream boundary conditions. The dataset provides a collection of pairs from the shape of the airfoil to the solution.

- **Transport equation** benchmark that represents a non-linear PDE, given by

$$\begin{cases} \frac{\partial u}{\partial t} + v \cdot \nabla u = 0 & \text{in } D \times (0, T) \\ u(x, 0) = f(x) & \text{in } D \\ v = [0.2, 0.2]^T \end{cases}$$

The dataset provides a collection of pairs $(f^{(i)}, u^{(i)})_i$ where $u^{(i)}$ are the evaluation of the solution operator $\mathcal{G} : f \rightarrow u(\cdot, T)$ that maps the initial condition f into the solution

at the final time $T = 1$. Two different benchmarks are provided for this PDE, one with smooth initial data and one with discontinuous initial data.

- **Allen-Cahn equation** benchmark that represents a non-linear PDE, given by

$$\begin{cases} \frac{\partial u}{\partial t} = \Delta u - \varepsilon^2 u(u^2 - 1) & \text{in } D \times (0, T) \\ u(x, 0) = f(x) & \text{in } D \\ \varepsilon = 220 \end{cases}$$

The dataset provides a collection of pairs $(f^{(i)}, u^{(i)})_i$ where $u^{(i)}$ are the evaluation of the solution operator $\mathcal{G} : f \rightarrow u(\cdot, T)$ that maps the initial condition f into the solution at the final time $T = 0.0002$.

5.1 Effectiveness of the library

For all previous datasets, we train an FNO and a CNO with the choice of hyperparameters reported in the article [32, 33] and we will call them “**literature**” hyperparameters since they are the best found by the authors in the literature. Then we use the hyperparameter optimization workflow, finding our best as possible configuration, and we call it “**best**” hyperparameters. Since the hyperparameter optimization process pulls the hyperparameters to obtain larger and heavier architectures, we also run the hyperparameter optimization process fixing the number of trainable parameters to be equal to the “literature” configuration for a fair comparison, following Sect. 4.2. We denote these hyperparameter configurations with “**same dof**” hyperparameters.

For the FNO architecture, we optimize the learning rate ($\lambda \in [10^{-4}, 10^{-2}]$), the factor to reduce the learning rate ($\eta \in [0.75, 0.99]$), the regularization factor ($w \in [10^{-6}, 10^{-3}]$), the hidden dimension ($d_v \in \{4, 8, 16, 32, 64, 96, 128\}$), the number of hidden layers ($1 \leq L \leq 6$), the number of modes considered in the Fourier transform ($k_{max} \in \{2, 4, 8, 12, 16, 20, 24, 28, 32\}$), the activation function used ($\sigma \in \{GELU, LReLU, ReLU, tanh\}$ respectively the Gaussian Error Linear Units function [10], LeakyReLU function, Rectified Linear Units function and hyperbolic tangent function), the padding dimension ($0 \leq pad \leq 15$) and the Fourier architecture modification ($arc \in \{Classic, MLP, Residual\}$), see Sect. 2.5 and [30]. The resulting errors we get are in Table 1 and the hyperparameters found are in Table 2.

We mention that the best hyperparameters found for the Poisson problem for the FNO architecture (both with the unconstrained optimization and the constrained “same dofs” optimization) have only one layer ($L = 1$). At first glance, this may seem strange, but it is perfectly interpretable with spectral theory for solving the Poisson problem in the Fourier domain, and is consistent with the proof of the universal approximation theorem for the FNO architecture [13].

We performed the same experiment for the emerging CNO model. In this case, we select for optimization the core hyperparameters in the definition of the CNO architecture. In particular, we tune the learning rate ($\lambda \in [10^{-4}, 10^{-2}]$), the learning rate reduction factor ($\eta \in [0.75, 0.99]$), the regularization factor ($w \in [10^{-6}, 10^{-3}]$), the number of layers ($1 \leq L \leq 5$), the number of layers for the final residual block ($1 \leq N \leq 6$), the number of layers for the remaining residual blocks ($1 \leq M \leq 8$), the channel multiplier ($chan_mul \in \{8, 16, 24, 32, 40, 48\}$) and the kernel size ($k \in \{3, 5, 7\}$). The resulting errors are listed in Table 3 and the hyperparameters are listed in Table 4.

Table 1 The tests are performed for all the benchmarks previously described with the FNO architecture. We mark in bold the lower test error for each benchmark

Problem	Literature	Same dofs	Best
Poisson eq.	$(5.61 \pm 0.09)\%$	$(1.46 \pm 0.07)\%$	$(1.04 \pm 0.09)\%$
Darcy eq.	$(1.05 \pm 0.01)\%$	$(0.81 \pm 0.02)\%$	$(1.00 \pm 0.01)\%$
Navier-Stokes eq.	$(3.83 \pm 0.10)\%$	$(3.26 \pm 0.08)\%$	$(3.26 \pm 0.08)\%$
Wave eq.	$(1.27 \pm 0.05)\%$	$(0.95 \pm 0.10)\%$	$(1.08 \pm 0.04)\%$
Euler eq.	$(0.48 \pm 0.03)\%$	$(0.420 \pm 0.002)\%$	$(0.42 \pm 0.002)\%$
Smooth transport eq.	$(0.40 \pm 0.03)\%$	$(0.17 \pm 0.01)\%$	$(0.036 \pm 0.001)\%$
Discontinuous transport eq.	$(1.30 \pm 0.08)\%$	$(1.09 \pm 0.07)\%$	$(1.01 \pm 0.02)\%$
Allen-Cahn eq.	$(0.48 \pm 0.05)\%$	$(0.22 \pm 0.01)\%$	$(0.25 \pm 0.028)\%$

In the table, we report the resulting mean L^1 errors obtained with the literature hyperparameters (second column), the optimized hyperparameter configuration (last column) and the hyperparameters with the constraint to keep the number of trainable parameters fixed and equal to the literature configuration (third column). For each test, we report the mean and the standard deviation after three trials
The bold values represent the lower test error achieved along the test set

For all the tests performed, we use the AdamW optimizer, reducing the learning rate by a factor of η every 10 epochs for the FNO architecture and every epoch for the CNO architecture. We adopt the relative mean L^1 as a loss function along all the tests. We evaluate 200 hyperparameter configurations, the time needed for each test strongly depends on the dimension of the datasets and the configuration of the search space, experiments take up to two days each on a single 4090 NVIDIA GPU.

5.2 Smooth transport equation example

Hyperparameter configurations comparison

The most interesting case is the transport equation with smooth initial data, where careful tuning of the hyperparameters of the FNO's architecture can lead to an order of magnitude improvement in the loss function, eliminating errors due to manual optimization using a heuristic hyperparameter optimization process based on grid search or random search algorithms, see Table 1. For better understanding, we examine two input functions where the "literature" parameter configuration results in the highest error. For each input function, we show the true output, the approximation of the FNO under the "literature" configurations, and the absolute error between the true output and the approximation of the FNO. For the same input functions, we also compute and plot the FNO's approximation using the optimized hyperparameter configuration obtained through our optimization process, alongside the corresponding absolute error in Fig. 4. The plots indicate that the optimized hyperparameter configuration outperforms the "literature" configuration in all examples, with more spread and smaller absolute values of the error. We present two representative examples. All trained models and evaluations are available at the following website <https://hypernos.streamlit.app/>.

Furthermore, we plot the loss functions for both the train set and the test set, comparing the results for the default hyperparameter configuration and the optimized (best) configuration. We also provide a plot showing the distribution of the error over the test set, see Fig. 5. From Fig. 5a we notice that the training loss function with the optimized hyperparameters

Table 2 Obtained hyperparameters for FNO architecture across all the benchmarks

Problem	λ	η	w	d_v	L	k_{max}	σ	pad	arc
Best	Poisson eq.	0.0005	0.1 · 10 ⁻⁴	128	1	32	tanh	0	MLP
	Darcy eq.	0.0003	0.8 · 10 ⁻⁴	32	4	32	LReLU	0	Classic
	Navier-Stokes eq.	0.00308	6.75 · 10 ⁻⁴	64	5	20	LReLU	0	MLP
	Wave eq.	0.00173	0.03 · 10 ⁻⁴	32	4	28	GELU	1	Classic
	Euler eq.	0.008	3.45 · 10 ⁻⁴	32	3	8	GELU	9	MLP
	Smooth transport eq.	0.00298	7.51 · 10 ⁻⁴	128	5	28	LReLU	5	Classic
	Disc. transport eq.	0.00681	1.77 · 10 ⁻⁴	64	4	32	tanh	0	Residual
	Allen-Cahn eq.	0.00275	2.38 · 10 ⁻⁴	8	2	28	ReLU	3	Classic
	Poisson eq.	0.00721	0.12 · 10 ⁻⁴	8	1	16	LReLU	2	Classic
	Darcy eq.	0.00104	2.5 · 10 ⁻⁴	32	3	24	GELU	14	MLP
Same dofs	Wave eq.	0.00223	7.11 · 10 ⁻⁴	16	3	20	GELU	6	Classic
	Smooth transport eq.	0.0081	9.07 · 10 ⁻⁴	16	5	20	LReLU	5	MLP
	Disc. transport eq.	0.00852	5.86 · 10 ⁻⁴	32	3	16	tanh	5	Residual
	Allen-Cahn eq.	0.00702	6.5 · 10 ⁻⁴	16	1	20	LReLU	7	Classic

Where the hyperparameter for the “same dofs” is not specified means that they are equal to the “best” hyperparameters

Table 3 The tests are performed for all the benchmarks previously described with the CNO architecture. We mark in bold the lower test error for each benchmark

Problem	Literature	Same dofs	Best
Poisson eq.	$(0.45 \pm 0.02)\%$	$(0.30 \pm 0.04)\%$	$(0.31 \pm 0.036)\%$
Darcy eq.	$(0.67 \pm 0.01)\%$	$(0.56 \pm 0.04)\%$	$(0.56 \pm 0.04)\%$
Navier-Stokes eq.	$(3.67 \pm 0.17)\%$	$(3.57 \pm 0.11)\%$	$(3.45 \pm 0.15)\%$
Wave eq.	$(1.54 \pm 0.02)\%$	$(1.38 \pm 0.10)\%$	$(0.96 \pm 0.06)\%$
Euler eq.	$(0.43 \pm 0.01)\%$	$(0.391 \pm 0.001)\%$	$(0.390 \pm 0.001)\%$
Smooth transport eq.	$(0.84 \pm 0.04)\%$	$(0.47 \pm 0.05)\%$	$(0.42 \pm 0.05)\%$
Discontinuous transport eq.	$(1.41 \pm 0.01)\%$	$(1.21 \pm 0.05)\%$	$(1.21 \pm 0.03)\%$
Allen-Cahn eq.	$(2.06 \pm 0.25)\%$	$(1.43 \pm 0.10)\%$	$(1.41 \pm 0.03)\%$

In the table, we report the resulting mean L^1 errors obtained with the literature hyperparameters (second column), the optimized hyperparameter configuration (last column) and the hyperparameters with the constraint to keep the number of trainable parameters fixed and equal to the literature configuration (third column). For each test, we report the mean and the standard deviation after three trials

The bold values represent the lower test error achieved along the test set

decreases faster and with less oscillation at the end of the training process. We can also see from Fig. 5b that the error of the model with optimized hyperparameters is consistently one order of magnitude lower than the error of the model with the default configuration. This comprehensive comparison highlights the improved accuracy and performance achieved by the optimized hyperparameters consistently across all quantities considered.

The (5b), Fig. 6, shows the loss functions for all hyperparameter configurations tested during the optimization process for the FNO model approximating the operator associated with the transport equation with smooth data. The plot has on the y-axis the relative L^1 error and on the x-axis the time (the whole simulation takes about 27 hours), and tries with 150 different hyperparameter configurations. In other words, this plot represents the best relative error in the loss function that we get as the simulation time increases. The first loss plotted is the loss relative to the default hyperparameter configuration, i.e. the first hyperparameter configuration tested, as mentioned in Sect. 4.1. For this test, we use a single GPU but run two trials in parallel, which explains why there are two overlapping losses at the same time. The *width* of a single loss is an indicator of the weight and size of the model. We can also see that the best configuration is obtained almost at the end of the optimization process, but near-optimal results are already obtained in the first half of the simulation.

Configurations with matched complexity

From the previous comparison (Table 2) we observe that the optimized (unconstrained) model has a very large number of trainable parameters: approximately 250'000'000 (about 250M) compared with roughly 8'000'000 (8M) for the “literature” model. To assess how much improvement is achievable at fixed complexity, we apply the routine described in Sec. 4.2; the resulting performance for the constrained setting is reported in Table 2. In this example, the performance can be improved by about a factor of two even when the number of trainable parameters is held fixed, while fully exploiting the potential requires exploring substantially larger and more expressive models. We therefore further study several constrained regimes and report in Table 5 the previous state-of-the-art results (“literature”), the best configuration with the same number of trainable parameters (“same dofs”), the

Table 4 Optimized hyperparameters for CNO architecture across all the benchmarks. Where the hyperparameter for the “same dofs” is not specified means that they are equal to the “best” hyperparameters

	Problem	λ	η	w	L	N	M	chan_mul	ker_size
best	Poisson eq.	0.00595	0.97	$8.18 \cdot 10^{-4}$	3	1	2	16	5
	Darcy eq.	0.0056	0.99	$8.8 \cdot 10^{-4}$	3	3	3	40	3
	Navier-Stokes eq.	0.00218	0.99	$5.17 \cdot 10^{-4}$	4	3	7	56	3
	Wave eq.	0.00282	0.99	$0.51 \cdot 10^{-4}$	4	3	7	32	5
	Euler eq.	0.00377	0.99	$4.56 \cdot 10^{-4}$	3	5	2	48	3
	Smooth transport eq.	0.00211	0.99	$8.83 \cdot 10^{-4}$	2	5	4	32	7
	Disc. transport eq.	0.00622	0.99	$3.0 \cdot 10^{-4}$	3	3	1	56	3
same dofs	Allen-Cahn eq.	0.0016	0.99	$4.25 \cdot 10^{-4}$	3	3	4	48	3
	Poisson eq.	0.00147	0.98	$4.81 \cdot 10^{-4}$	2	2	6	40	5
	Navier-Stokes eq.	0.00129	0.99	$2.3 \cdot 10^{-4}$	3	3	6	23	5
	Wave eq.	0.00369	0.99	$3.6 \cdot 10^{-4}$	4	4	3	27	3
	Smooth transport eq.	0.00138	0.99	$4.6 \cdot 10^{-4}$	3	3	5	33	3
	Disc. transport eq.	0.0093	0.99	$3.8 \cdot 10^{-5}$	4	1	4	17	3
	Allen-Cahn eq.	0.00084	0.99	$2.7 \cdot 10^{-4}$	3	2	4	32	3

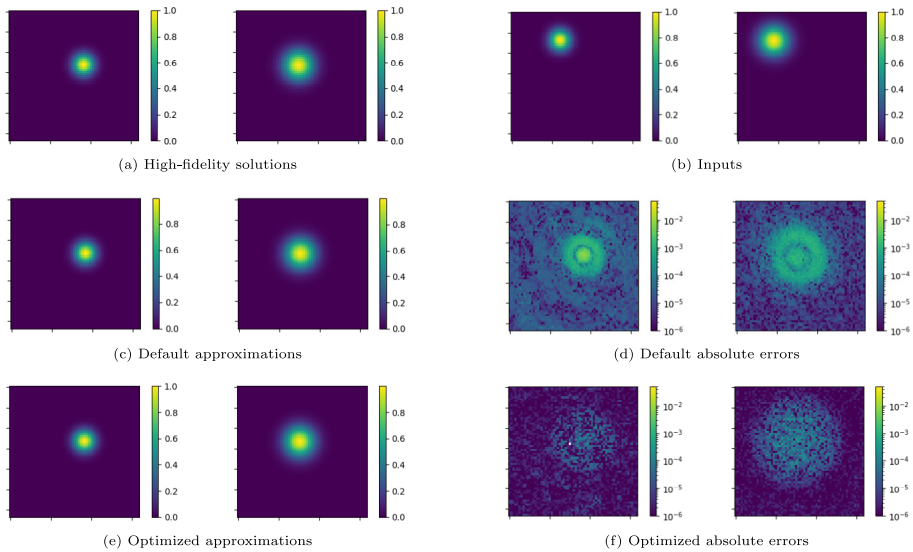


Fig. 4 Comparison of FNO approximations for the transport equation with smooth initial data using “literature” and “best” hyperparameter configurations. The first row shows the high-fidelity solutions 4a and the corresponding inputs 4b. The second row displays the FNO approximations 4c and absolute errors 4d using the “literature” hyperparameter configuration, while the third row presents the FNO approximations 4e and absolute errors 4f using the optimized “best” hyperparameter configuration. The optimized configuration consistently yields lower absolute errors across all test cases

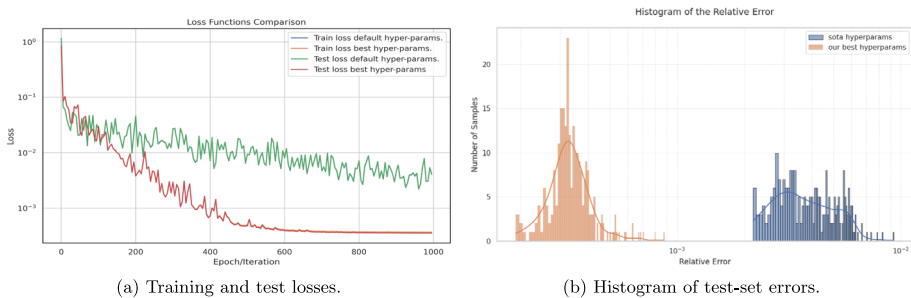


Fig. 5 Relative L^1 error for the default and optimized hyperparameter configurations: (5a) training and test loss curves; (5b) histogram of test-set errors. In (5a), the training and test curves nearly overlap and therefore appear as a single line. The optimized hyperparameter configuration leads to a lower training loss and a lower test loss. Furthermore, the error distribution of the optimized hyperparameter configuration is consistently shifted to the left compared to the default configuration, indicating a lower error across the entire test set (5b)

best unconstrained configuration (“best”), and the best configurations obtained under upper bounds of 500'000, 50'000'000 and 150'000'000 parameters (“best 500k”, “best 50M”, “best 150M” respectively). The corresponding hyperparameters for the new constrained configurations are listed in Table 6. Finally, Fig. 7 shows the convergence of the test relative L^1 error as a function of the number of trainable parameters; the error decreases monotonically with increasing model capacity. This trend is consistent with an empirical convergence rate of order $N^{-0.2}$ (slope -0.2 in the log–log plot), where N denotes the number of trainable parameters, quantifying the efficiency of added capacity.

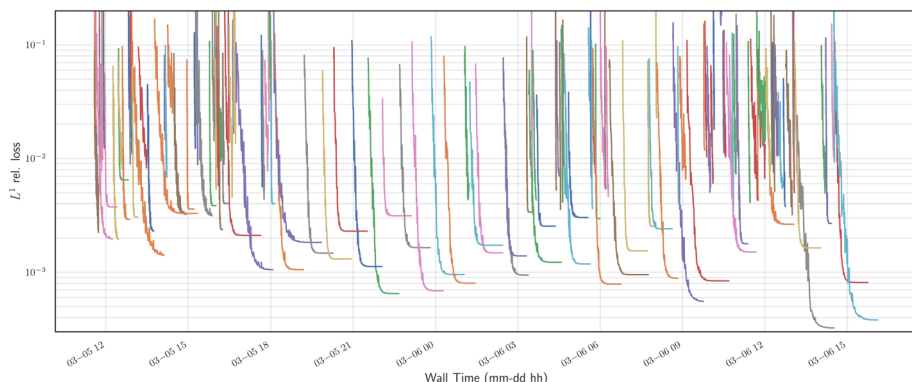


Fig. 6 Relative L^1 error obtained by each evaluated hyperparameter configuration as a function of elapsed wall-clock time from the start of the optimization. The trajectory illustrates the progressive improvement in the best observed error over the course of the search

Table 5 Comparison of the relative L^1 error for the FNO approximating the transport equation with smooth initial data for different constraints on the number of trainable parameters. We mark in bold the lower test error for each benchmark

literature	best 500k	same dofs (8M)	best 50M	best 150M	best (250M)
$(0.40 \pm 0.03)\%$	$(0.29 \pm 0.01)\%$	$(0.17 \pm 0.01)\%$	$(0.078 \pm 0.007)\%$	$(0.052 \pm 0.003)\%$	$(0.036 \pm 0.001)\%$

The “literature” column refers to the results obtained with the hyperparameters from [32, 33], “same dofs” refers to the best configuration with the same number of trainable parameters as the literature configuration, and “best 500k”, “best 50M”, “best 150M” and “best” refer to the best configurations obtained under upper bounds of 500’000, 50’000’000, 150’000’000 and no constraint on the number of trainable parameters, respectively. For each test, we report the mean and the standard deviation after three trials
The bold values represent the lower test error achieved along the test set

Table 6 Values of the hyperparameters for FNO architecture on the transport equation with continuous initial data

problem	λ	η	w	d_v	L	k_{max}	σ	pad	arc
best 500k	0.00455	0.93	$7.0 \cdot 10^{-4}$	16	4	11	LReLU	7	Classic
best 50M	0.0078	0.89	$0.67 \cdot 10^{-4}$	64	5	24	LReLU	12	Classic
best 150M	0.00874	0.89	$1.12 \cdot 10^{-4}$	48	5	32	LReLU	15	Classic

In particular, we report the hyperparameters found for the new configurations; the others are reported in Table 2

6 Conclusions

The HyperNOs library provides a scalable framework tool for the study of neural operators, providing a powerful and scalable framework for automated hyperparameter optimization. By combining parallel computing with state-of-the-art optimization algorithms, HyperNOs enables researchers and practitioners to efficiently tune the hyperparameters of neural operators. This is useful for unlocking the full potential of neural operators to solve complex real-world problems, for exhaustively exploring the properties of neural operators, and for fair and comprehensive comparison of different architectures and models. The library’s user-friendly interface, high flexibility in both data set and model definition, high-level func-

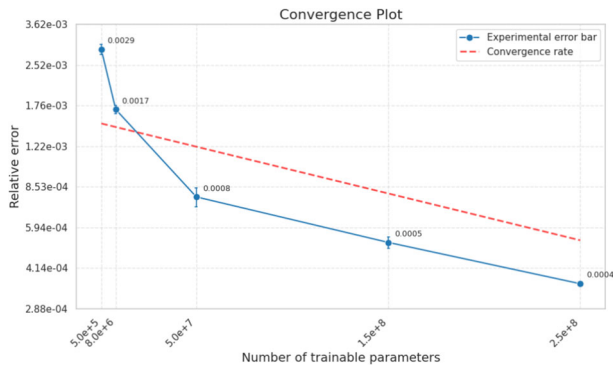


Fig. 7 Convergence of the test relative L^1 error for the FNO approximating the transport equation with smooth initial data as a function of the number of trainable parameters. Numbers adjacent to each marker denote the standard deviation of the error across trials. The reference guideline shows an empirical convergence rate of approximately 0.2, consistent with the observed decay

tionality, and integration with popular neural operator frameworks make it a useful resource for advancing research and applications in this rapidly evolving field. By automating this critical aspect of model development, researchers and practitioners can focus on higher-level tasks such as designing novel architectures, interpreting results, and applying neural operators to solve new and challenging real-world problems.

In future work, we aim to expand the range of implemented architectures available in the library, thereby increasing its versatility and applicability to a wider range of problems. This expansion will include the integration of state-of-the-art neural operator architectures, such as deep operator network, graph neural operators and attention-based models, which have shown significant success in various domains. To further support users of the library, we intend to introduce new functionalities that streamline the development and deployment of neural operator models. Through these enhancements, we aim to provide a comprehensive, user-friendly platform, that not only meets the current needs of the neural operator and deep learning community, but also anticipates and adapts to future advances in the field. By fostering collaboration and encouraging contributions from the open-source community, we aim to ensure that the library remains at the forefront of innovation, continuously evolving to meet the challenges and opportunities of modern machine learning.

Supplementary information

- We provide a web application that allows users to interact with the library results and visualize it for all the described models across the test sets of all the benchmarks. The web application is available at <https://hypernos.streamlit.app/> and the source code is available at https://github.com/MaxGhi8/HyperNOs_website.
- We create a video for the visualization of the Fourier Neural Operator architecture for both the one-dimensional and two-dimensional case. The video is available at <https://zenodo.org/records/14884675> [9] and the source code is available at <https://github.com/MaxGhi8/ManimAnimation/tree/main/FNO>.

Acknowledgements The author would like to thank Siddhartha Mishra for useful discussions, Giorgio Abelli for valuable support in refactoring some parts of the library and the reviewers for their insightful comments.

Author Contributions Massimiliano Ghiotto: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data Curation, Writing - Original Draft, Writing - Review & Editing, Visualization.

Funding Open access funding provided by Università degli Studi di Pavia within the CRUI-CARE Agreement.

Data Availability The data sets used in this study are available at <https://zenodo.org/records/10406879> [32, 33]; all the trained models are made public on Zenodo at <https://zenodo.org/records/15055547> [8].

Code Availability The library is public on GitHub and can be downloaded at <https://github.com/MaxGhi8/HyperNOs> following the related instructions.

Declarations

Financial interests All authors certify that they have no affiliations with or involvement in any organization or entity with any financial interest or non-financial interest in the subject matter or materials discussed in this manuscript.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

1. Backmeyer, M., Kurz, S., Möller, M., Schöps, S.: Solving electromagnetic scattering problems by isogeometric analysis with deep operator learning. In: 2024 Kleinheubach Conference, pp. 1-4. IEEE, (2024)
2. Bartolucci, F., de Bézenac, E., Raonic, B., Molinaro, R., Mishra, S., Alaifari, R.: Representation equivalent neural operators: a framework for alias-free operator learning. *Adv Neural Info Process Syst* 36, 69661–69672 (2024)
3. Bergstra, J., Bardenet, R., Bengio, Y., Kégl, B.: Algorithms for hyper-parameter optimization. *Adv Neural Info Process Syst* 24, (2011)
4. Canuto, C., Hussaini, M.Y., Quarteroni, A., Zang, T.A.: *Spectral Methods*, vol. 285. Springer, Berlin (2006)
5. Cao, S.: Choose a transformer: Fourier or galerkin. *Adv. Neural. Inf. Process. Syst.* 34, 24924–24940 (2021)
6. De Ryck, T., Mishra, S.: Numerical analysis of physics-informed neural networks and related models in physics-informed machine learning. *arXiv preprint arXiv:2402.10926* (2024)
7. Fanaskov, V. S., Oseledets, I. V.: 'Spectral neural operators. In: *Doklady Mathematics*, vol. 108, no. Suppl 2, pp. S226–S232. Moscow: Pleiades Publishing, (2023)
8. Ghiotto, M.: CNO and FNO trained model across the representative PDE benchmarks. Zenodo, March 20, (2025). <https://doi.org/10.5281/zenodo.15055547>
9. Ghiotto, M.: Visual representation of the Fourier neural operator architecture in 1D and 2D. Zenodo, (2025). <https://doi.org/10.5281/zenodo.14884675>
10. Hendrycks, D., Gimpel, K.: Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415* (2016)
11. Herde, M., Raonic, B., Rohner, T., Käppeli, R., Molinaro, R., de Bézenac, E., Mishra, S.: Poseidon: efficient foundation models for PDEs. *arXiv preprint arXiv:2405.19101* (2024)
12. Hughes, T.J.R., Cottrell, J.A., Bazilevs, Y.: Isogeometric analysis: CAD, finite elements, NURBS, exact geometry and mesh refinement. *Comput. Methods Appl. Mech. Eng.* 194(39–41), 4135–4195 (2005)
13. Kovachki, N., Lanthaler, S., Mishra, S.: On universal approximation and error bounds for Fourier neural operators. *J. Mach. Learn. Res.* 22(1), 13237–13312 (2021)
14. Kovachki, N., Li, Z., Liu, B., Azizzadenesheli, K., Bhattacharya, K., Stuart, A., Anandkumar, A.: Neural operator: Learning maps between function spaces with applications to pdes. *J. Mach. Learn. Res.* 24(89), 1–97 (2023)

15. Lanthaler, S., Mishra, S., Karniadakis, G.E.: Error estimates for deeponets: a deep learning framework in infinite dimensions. *Trans. Math. Appl.* **6**(1), tnac001 (2022)
16. Lanthaler, S., Li, Z., Stuart, A. M.: The nonlocal neural operator: Universal approximation. *arXiv preprint [arXiv:2304.13221](https://arxiv.org/abs/2304.13221)* (2023)
17. Li, B., Wang, H., Feng, S., Yang, X., Lin, Y.: Solving seismic wave equations on variable velocity models with Fourier neural operator. *IEEE Trans. Geosci. Remote Sens.* **61**, 1–18 (2023)
18. Li, Z., Zheng, H., Kovachki, N., Jin, D., Chen, H., Liu, B., Azizzadenesheli, K., Anandkumar, A.: Physics-informed neural operator for learning partial differential equations. *ACM/JMS J. Data Sci.* **1**(3), 1–27 (2024)
19. Li, L., Jamieson, K., Rostamizadeh, A., Gonina, E., Ben-Tzur, J., Hardt, M., Recht, B., Talwalkar, A.: A system for massively parallel hyperparameter tuning. *Proc. Mach. Learn. Syst.* **2**, 230–246 (2020)
20. Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Stuart, A., Bhattacharya, K., Anandkumar, A.: Multipole graph neural operator for parametric partial differential equations. *Adv. Neural. Inf. Process. Syst.* **33**, 6755–6766 (2020)
21. Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., Anandkumar, A.: Neural operator: graph kernel network for partial differential equations. *arXiv preprint [arXiv:2003.03485](https://arxiv.org/abs/2003.03485)* (2020)
22. Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., Anandkumar, A.: Fourier neural operator for parametric partial differential equations. *arXiv preprint [arXiv:2010.08895](https://arxiv.org/abs/2010.08895)* (2020)
23. Li, Z., Kovachki, N., Choy, C., Li, B., Kossai, J., Otta, S., Nabian, M. A. et al.: Geometry-informed neural operator for large-scale 3d pdes. *Adv. Neural Inf. Process. Syst.* **36**, 35836–35854 (2023)
24. Liu, Z., Wang, H., Zhang, H., Bao, K., Qian, X., Song, S.: Render unto numerics: orthogonal polynomial neural operator for PDEs with nonperiodic boundary conditions. *SIAM J. Sci. Comput.* **46**(4), C323–C348 (2024)
25. Lu, L., Jin, P., Pang, G., Zhang, Z., Karniadakis, G.E.: Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nat. Mach. Intell.* **3**(3), 218–229 (2021)
26. Marcati, C., Schwab, C.: Exponential convergence of deep operator networks for elliptic partial differential equations. *SIAM J. Numer. Anal.* **61**(3), 1513–1545 (2023)
27. Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., et al.: Ray: a distributed framework for emerging AI applications. In: 13th USENIX symposium on operating systems design and implementation (OSDI 18), pp. 561–577, (2018)
28. Pathak, J., Subramanian, S., Harrington, P., Raja, S., Chattopadhyay, A., Mardani, M., Kurth, T., et al.: FourCastNet: a global data-driven high-resolution weather model using adaptive Fourier neural operators. *arXiv preprint [arXiv:2202.11214](https://arxiv.org/abs/2202.11214)* (2022)
29. Pellegrini, L., Ghiotto, M., Centofanti, E., Pavarino, L. F.: Learning high-dimensional ionic model dynamics using Fourier neural operators. *arXiv preprint [arXiv:2505.14039](https://arxiv.org/abs/2505.14039)* (2025)
30. Qin, S., Lyu, F., Peng, W., Geng, D., Wang, J., Gao, N., Liu, X., Wang, L. L.: Toward a better understanding of fourier neural operators: analysis and improvement from a spectral perspective. *arXiv preprint [arXiv:2404.07200](https://arxiv.org/abs/2404.07200)* (2024)
31. Raissi, M., Perdikaris, P., Karniadakis, G.E.: Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* **378**, 686–707 (2019)
32. Raonic, B., Molinaro, R., Rohner, T., Mishra, S., de Bezenac, E.: Convolutional neural operators. In: *ICLR 2023 Workshop on Physics for Machine Learning*. (2023)
33. Raonic, B., et al.: Convolutional neural operators for robust and accurate learning of PDEs. *Adv. Neural. Inf. Process. Syst.* **36**, 77187–77200 (2023)
34. Ronneberger, O., Fischer, P., Brox, T.: U-net: convolutional networks for biomedical image segmentation. In: *Medical image computing and computer-assisted intervention-MICCAI 2015: 18th international conference, Munich, Germany, October 5–9, 2015, proceedings, part III 18*, pp. 234–241. Springer International Publishing, (2015)
35. Seidman, J., Kissas, G., Perdikaris, P., Pappas, G.J.: NOMAD: nonlinear manifold decoders for operator learning. *Adv. Neural. Inf. Process. Syst.* **35**, 5601–5613 (2022)
36. Tran, A., Mathews, A., Xie, L., Ong, C. S.: Factorized Fourier neural operators. *arXiv preprint [arXiv:2111.13802](https://arxiv.org/abs/2111.13802)* (2021)
37. Vetterli, M., Kovačević, J., Goyal, V.K.: *Foundations of Signal Processing*. Cambridge University Press (2014)
38. Wang, S., Wang, H., Perdikaris, P.: Learning the solution operator of parametric partial differential equations with physics-informed DeepONets. *Sci. Adv.* **7**(40), eabi8605 (2021)
39. Wen, G., Li, Z., Azizzadenesheli, K., Anandkumar, A., Benson, S.M.: U-FNO-An enhanced Fourier neural operator-based deep-learning model for multiphase flow. *Adv. Water Resour.* **163**, 104180 (2022)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.